# Mini-Project 2: 2.5D Scanner

Aditi Lagisetty, Bennett Macphee

## Introduction:

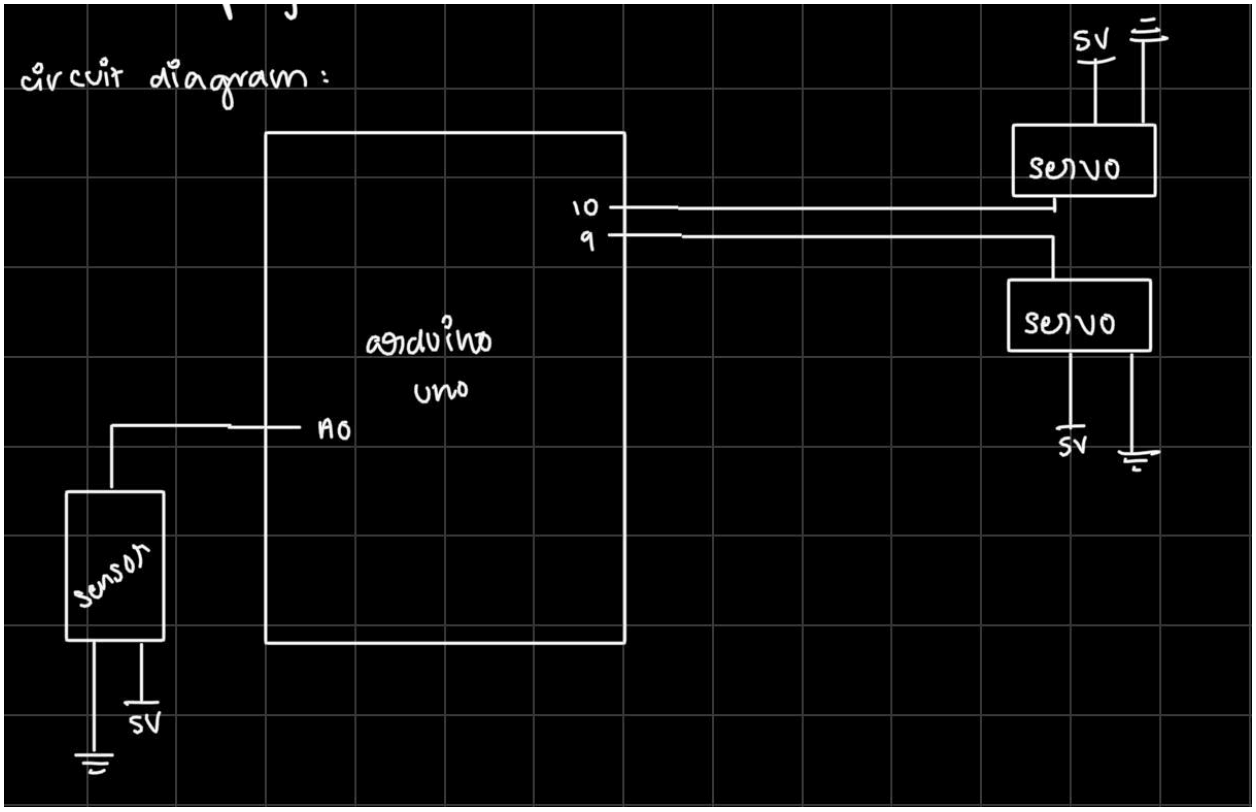
The goal of this project was to build a device that could scan a 2D object, in our case a cardboard cutout of a face, and plot it in a computer.

Our scanner used an Arduino Uno, a couple servos for movement and an IR sensor for the scanning. We started by designing a 3D printable structure for the scanner (Fig. 3) and assembling it. One requirement of the project was that we not use any irreversible assembly methods like glue, so all our physical parts are fastened with M3 hardware. The next step was to connect everything up and start calibrating the IR sensor. After that we moved on to testing and adjusting the code until we were getting reliable plots of our scans.

# System Design:

## Circuit Design:

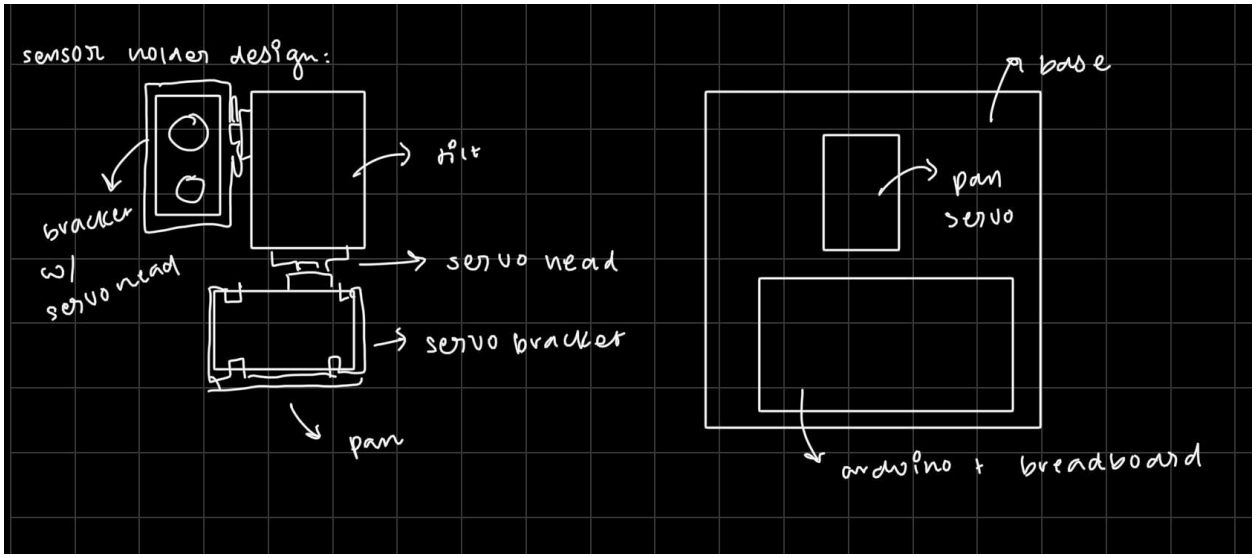
The circuit design for this mini project was very intuitive, essentially just connecting two servos to digital pins on the arduino and the sensor to an analog pin on the arduino. We made sure to have common ground and send power from the same rail to the servos and the sensor.



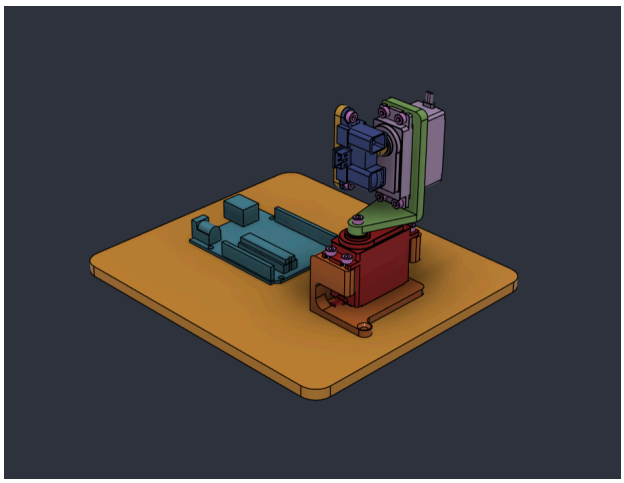

(Fig.1) Circuit diagram for our 2.5D scanner

## Mechanical:

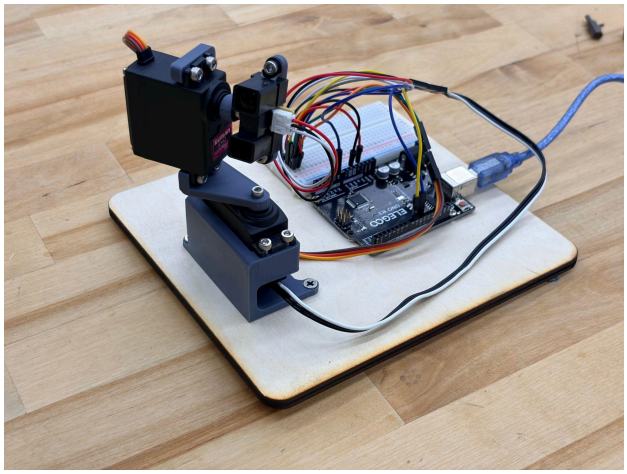
The driving factor behind our mechanical design of the scanner was to make both the scanner and code as simple as possible. We built the servo output splines into each part to avoid the extra complexity of servo horns, and we kept the two axes of rotation aligned with the center of the IR sensor which meant we didn't have to correct for any misalignment in code. The whole structure is made up of three printed parts and a laser cut plywood base. To us this felt like the minimum number of parts possible. Again in the simplicity department we designed around M3 x 8mm hardware all around which wasn't entirely necessary but made assembly easier.



(Fig.2) Drawing of our scanner design


(Fig.3) CAD model of our scanner

An issue we had was that one of our servos ended up spinning continuously when we gave it angle commands. Somebody probably did the old cut-off-the-potentiometer trick to turn it into a gear motor and threw it back in the bin when they were done. We untangled the wires and swapped the problematic servo out for a new one which worked fine.

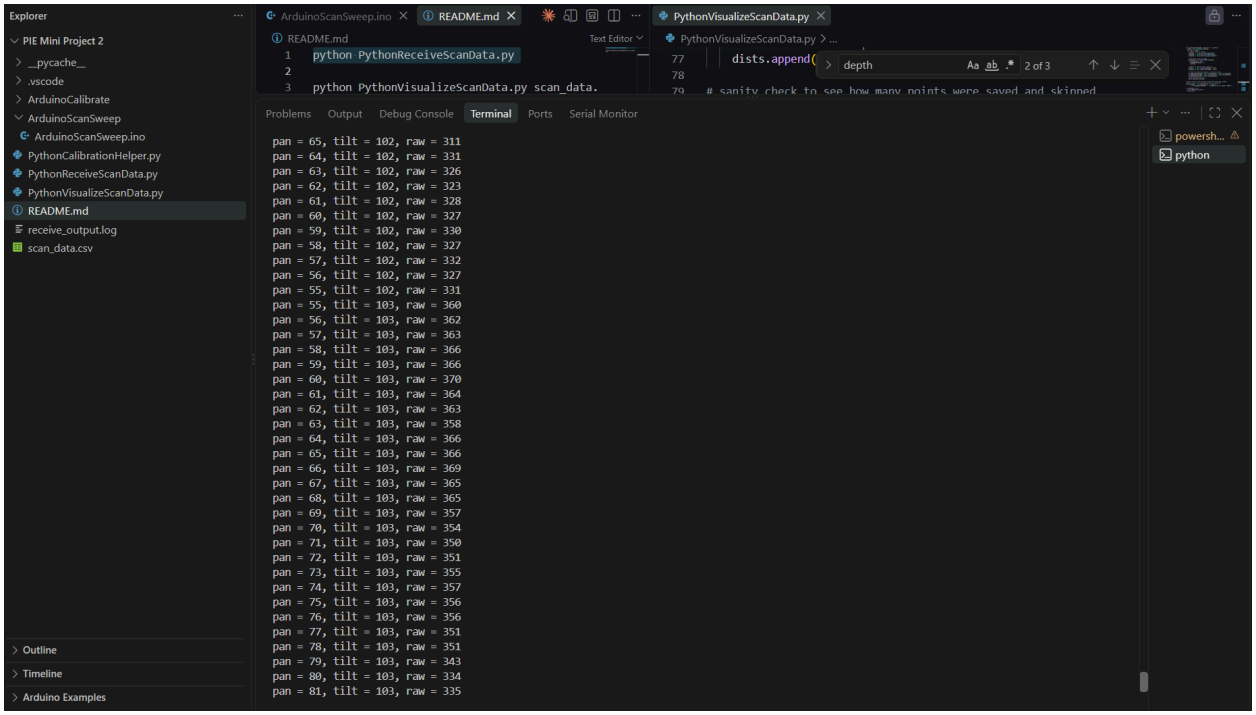


(Fig.4) Fully assembled.

## Software Design:

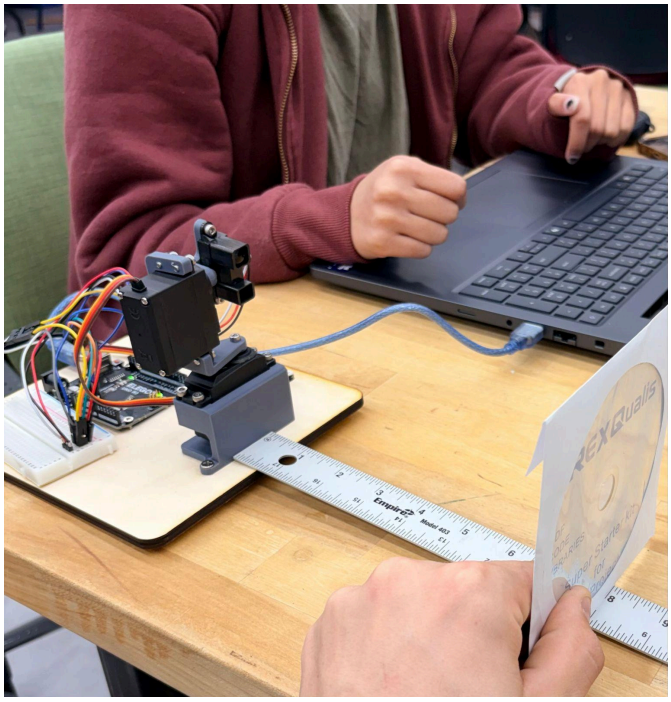
Our software pipeline moves data through four stages: an Arduino sketch sweeps the sensor across a pan/tilt grid and streams raw readings over serial, a Python script receives and logs that stream to CSV, a calibration script fits a raw-reading-to-distance curve from known reference distances, and a visualization script converts the logged data into a 2.5D point cloud. We implemented the following stages:

1) Scan (ArduinoScanSweep.ino): the Arduino drives the pan and tilt servos through a raster scan over a fixed angular range, sampling the sensor at each point and sending it to the laptop over serial.

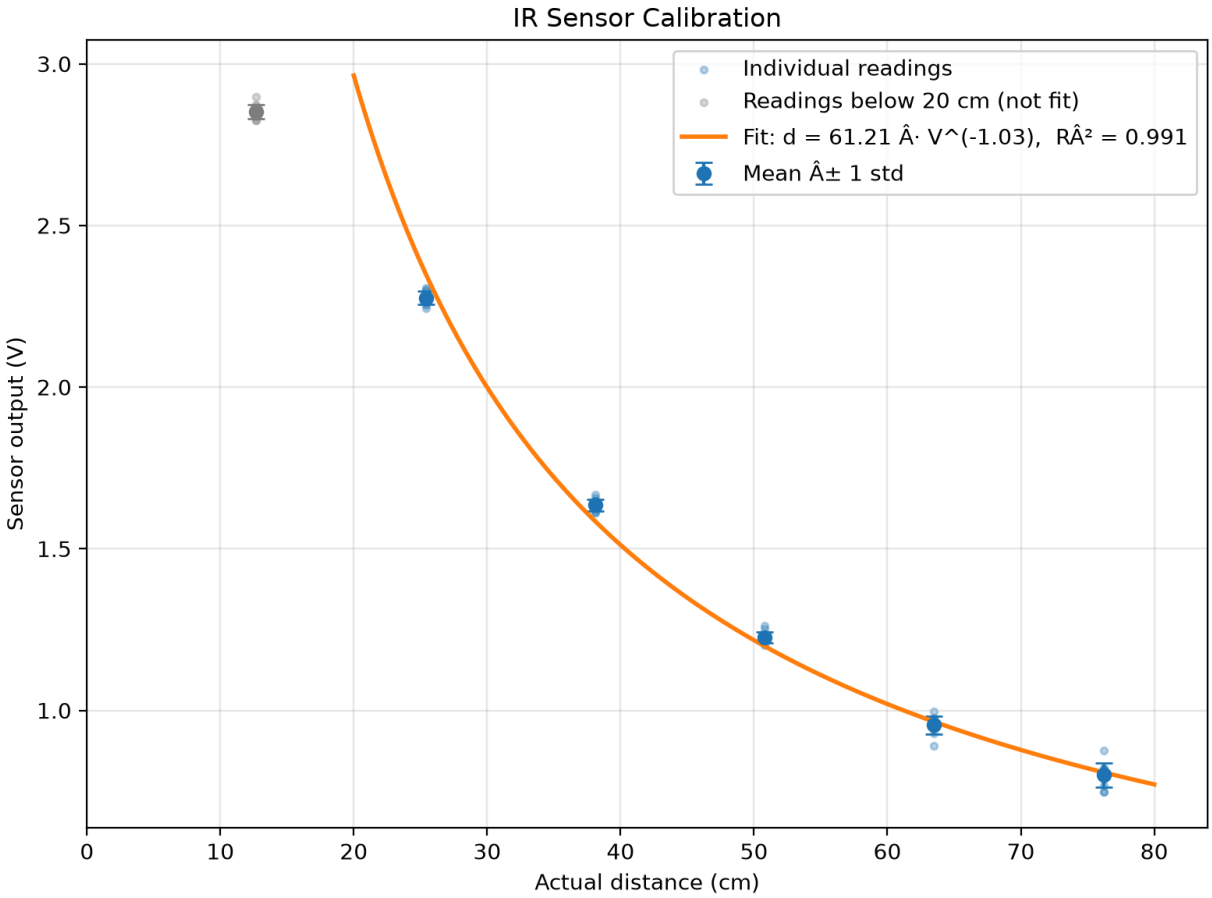



(Fig.5) Printing pan and tilt angles as well as raw ADC values.

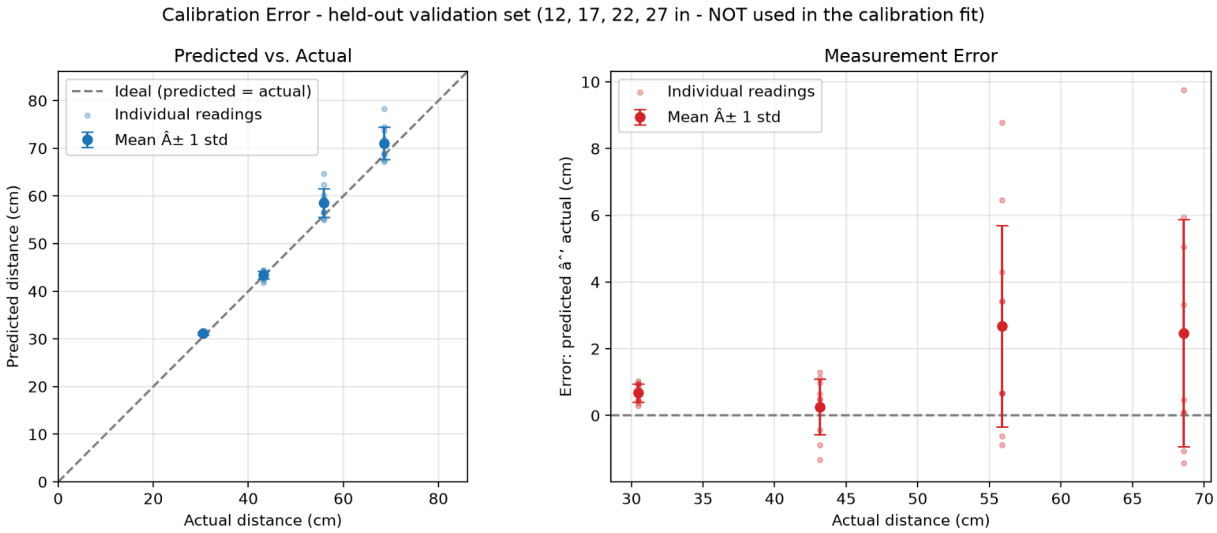
2) Receive (PythonReceiveScanData.py): the laptop reads each serial line, parses it into a (panAngle, tiltAngle, rawValue) tuple, and logs every point to scan_data.csv.
3) Calibrate (ArduinoCalibrate.ino + PythonCalibrationHelper.py): a separate sketch holds the sensor still and streams raw readings while the Python helper prompts us to place a target at known distances, then fits a power-law curve relating raw ADC value to distance.
   Before calibrating, we confirmed the sensor worked by running the Arduino AnalogInOutSerial example and watching the reading change as we moved a hand toward it



(Fig.6) Calibrating our IR sensor with known distances.

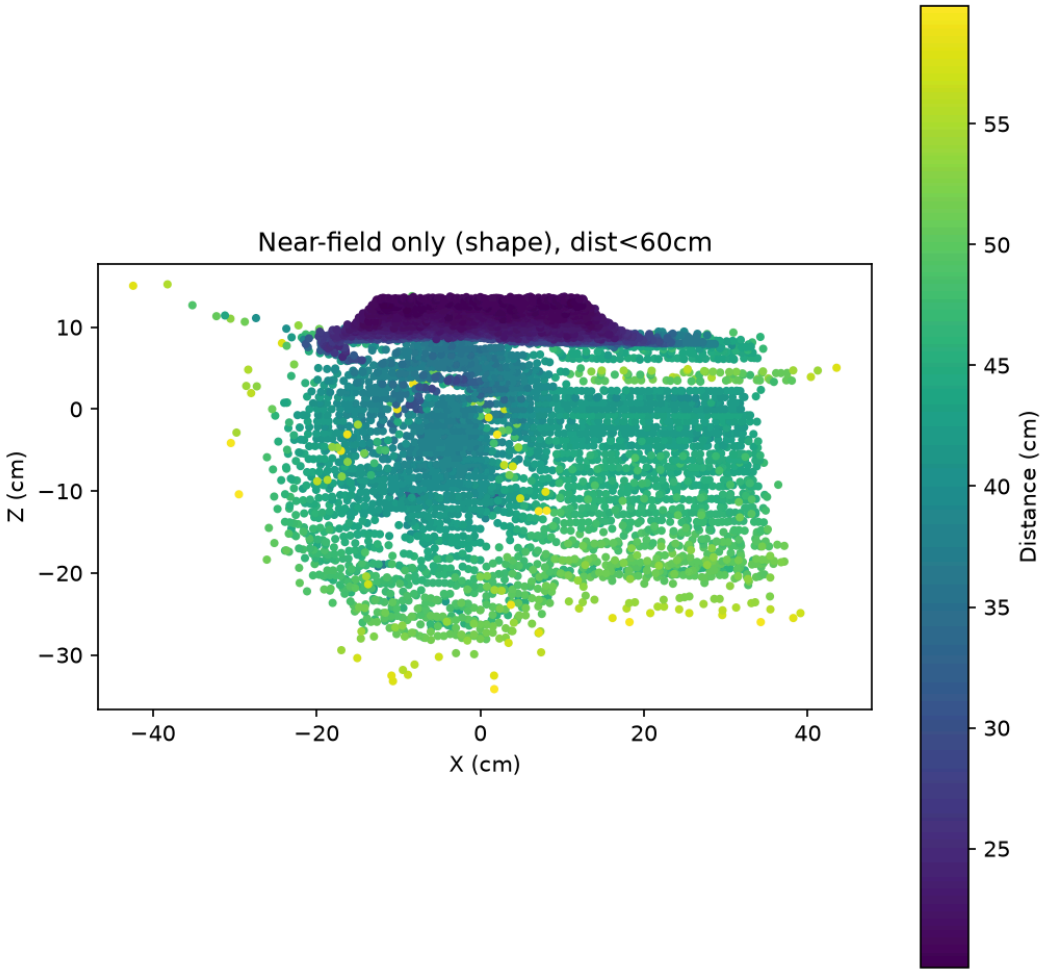



(Fig. 7) IR sensor calibration: sensor output voltage vs. actual distance, with 10 readings at each of six distances (12.7–76.2 cm). The 12.7 cm point (grey) is below the sensor's 20 cm rated minimum and was excluded from the fit. Power-law fit: d = 61.21·V^−1.03 ($R^2$ = 0.991).

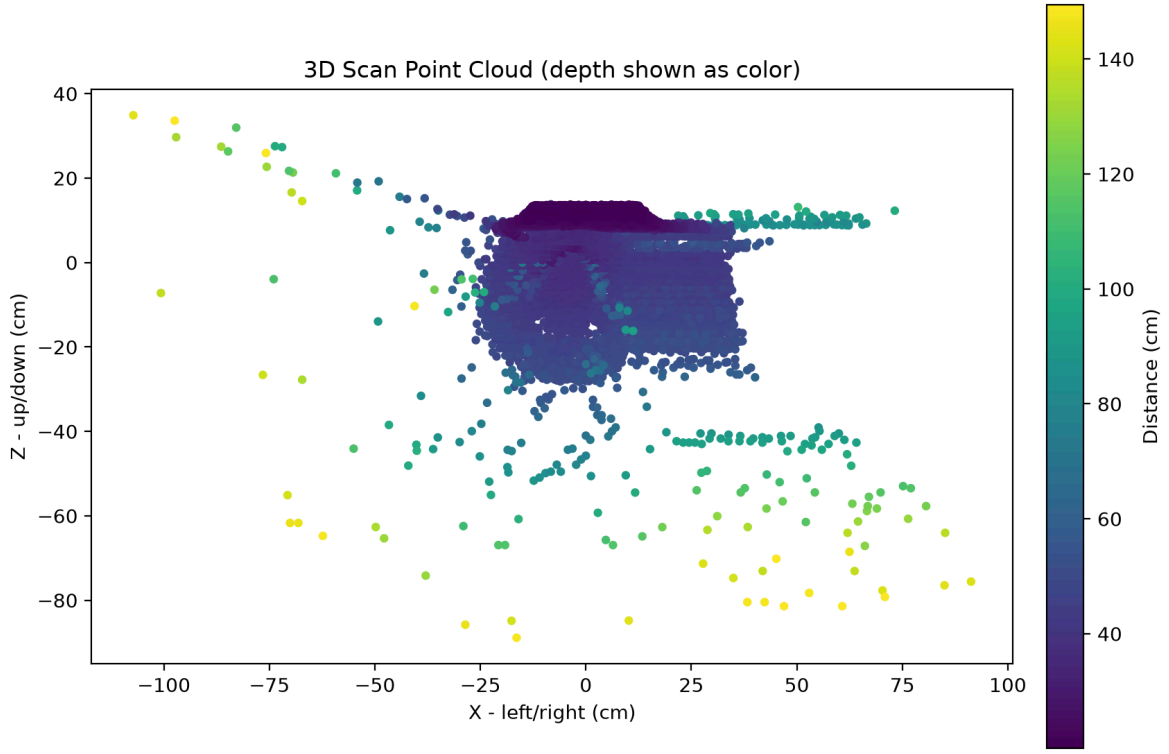



(Fig. 8) Calibration error on a held-out validation set (12, 17, 22, 27 in) not used in the fit. Left: predicted vs. actual distance. Right: error (predicted – actual). Mean error is under 1 cm at 30 and 43 cm but grows to about +2.5 cm at 56 and 69 cm, and individual readings there are off by up to ~9 cm, showing the sensor gets noisier with distance.

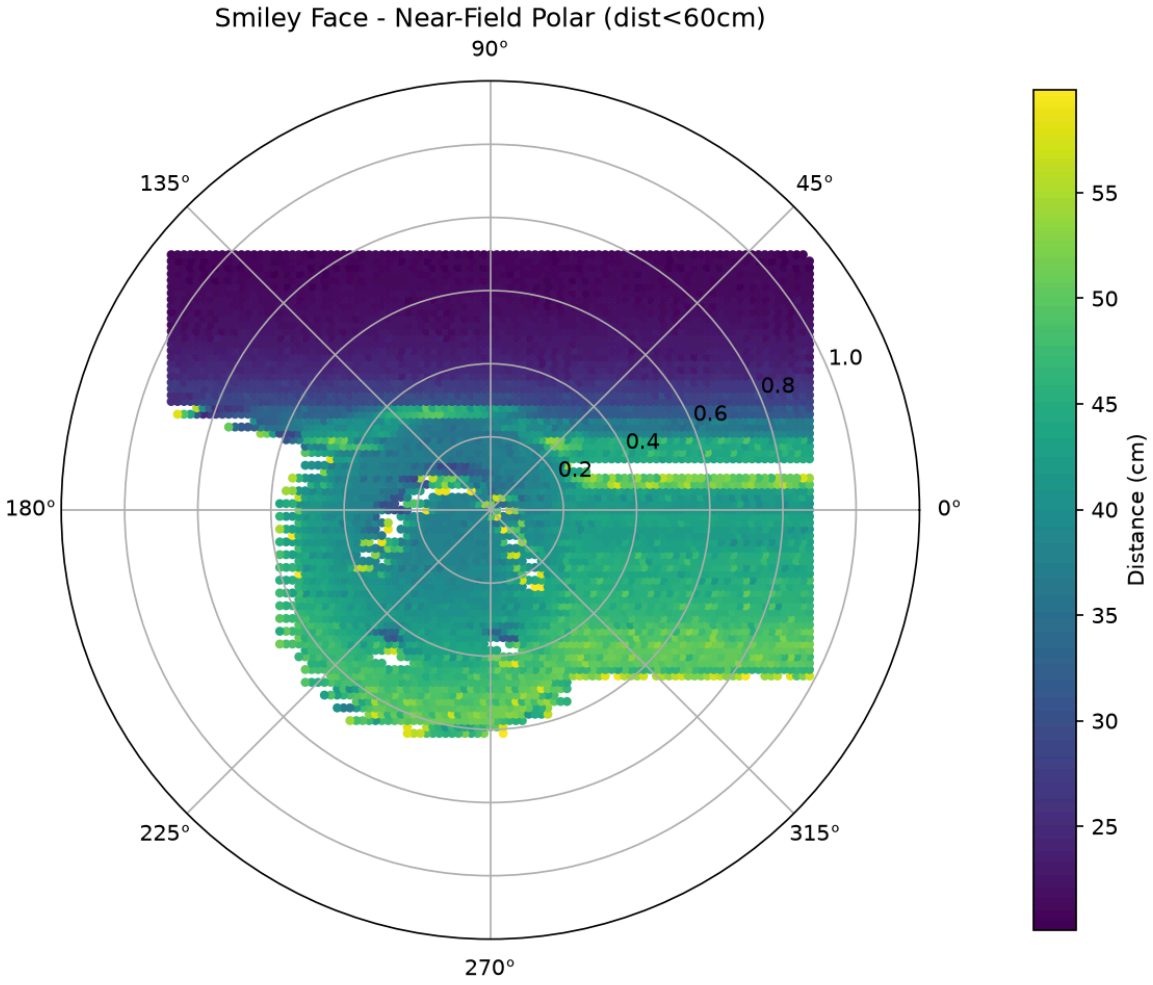
4) Visualize (PythonVisualizeScanData.py): the logged CSV is converted from raw sensor readings and servo angles into Cartesian coordinates and rendered as a point cloud.



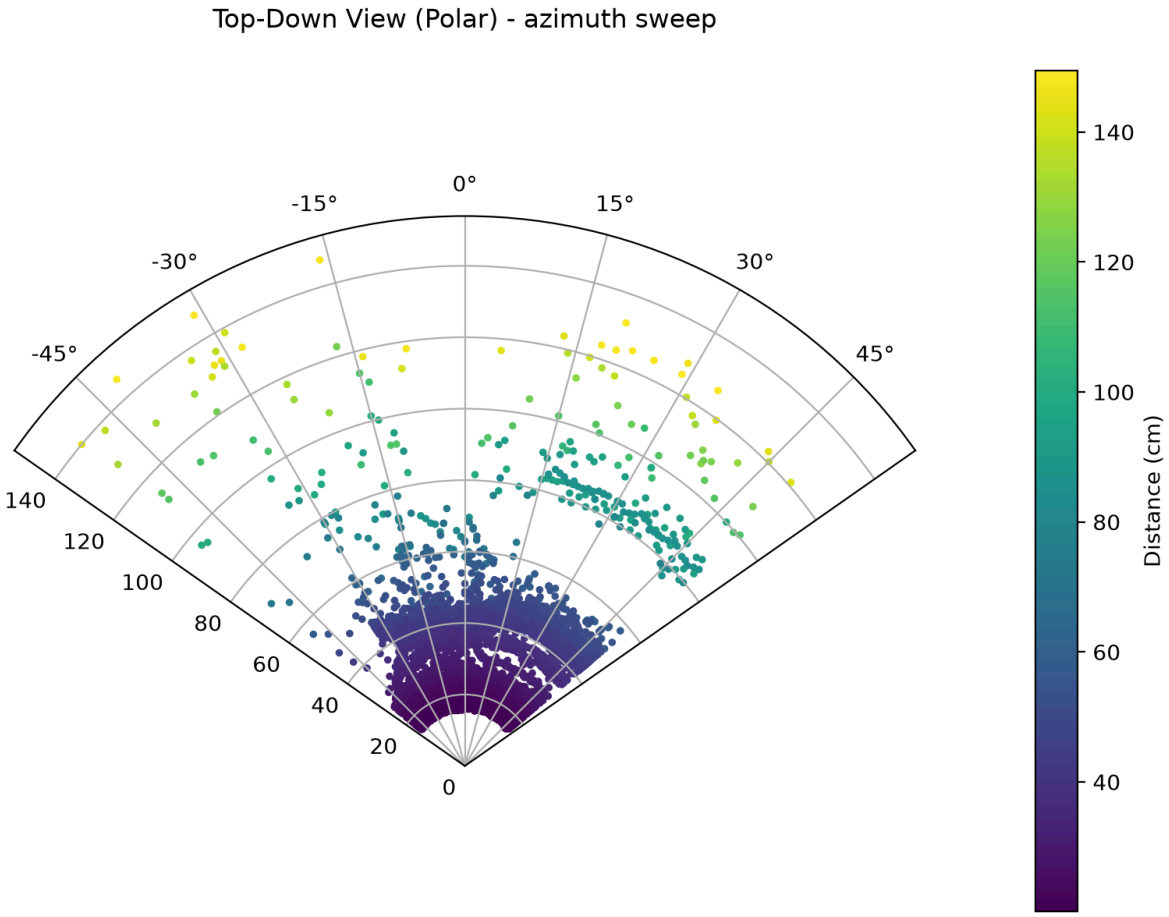

(Fig. 9) Front (X–Z) view of the smiley face scan, filtered to points closer than 60 cm and colored by distance.




(Fig. 10) Front view of all points in the sensor's valid range (20–150 cm). The scattered points outside the face are background surfaces.

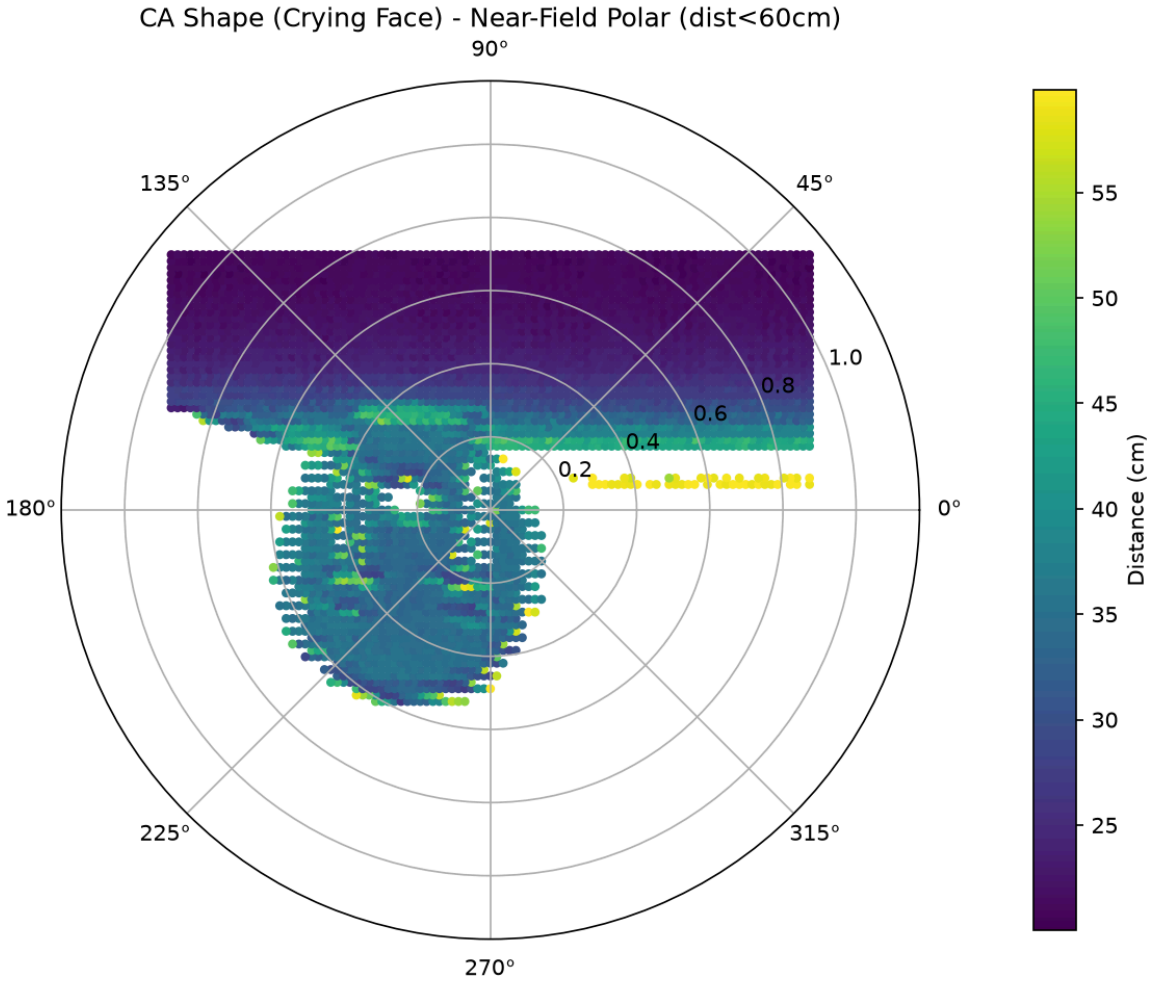



(Fig. 11) Front view of the smiley face in polar form: the angle is the direction around the scan center, the radius is the angular offset from center (radians), and color is distance. Filtered to under 60 cm.

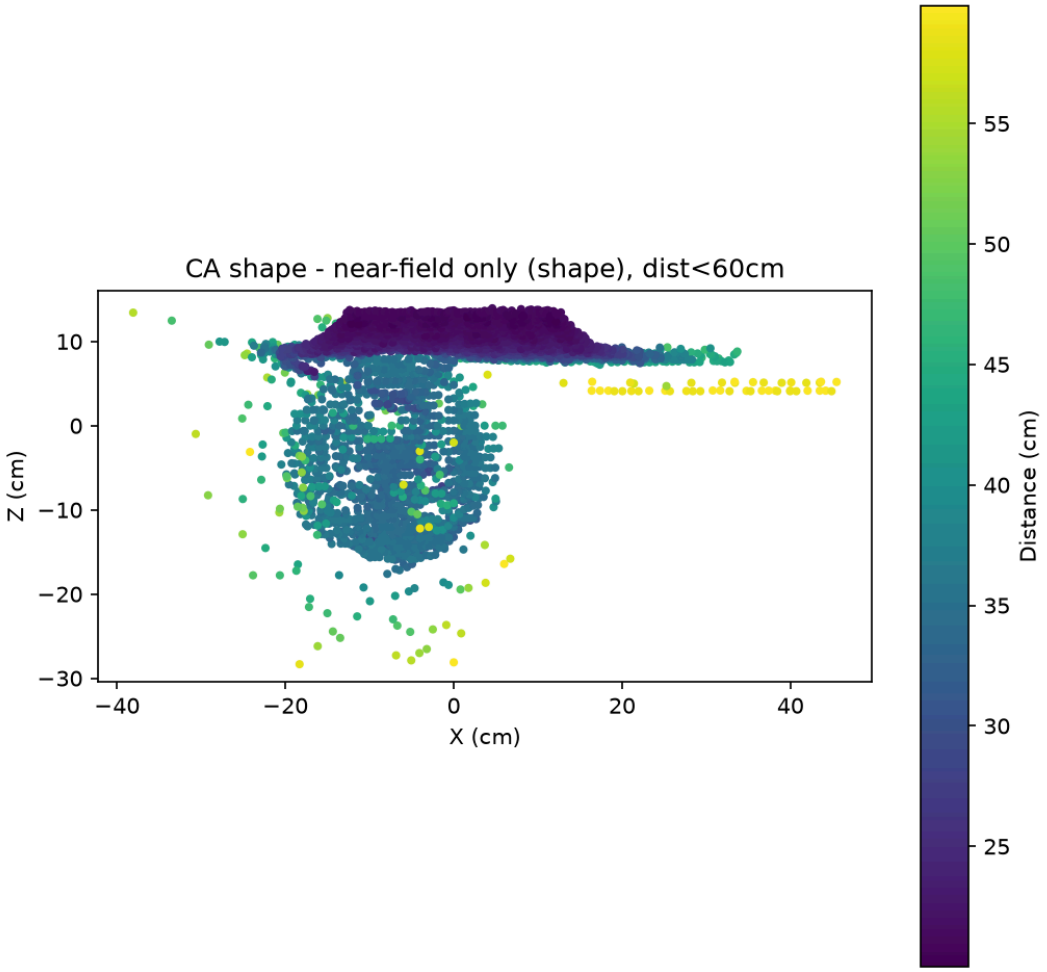



(Fig. 12) Top-down polar view of the smiley face scan. The face is the dense cluster at 30–45 cm; points beyond ~70 cm are the wall and room behind it.

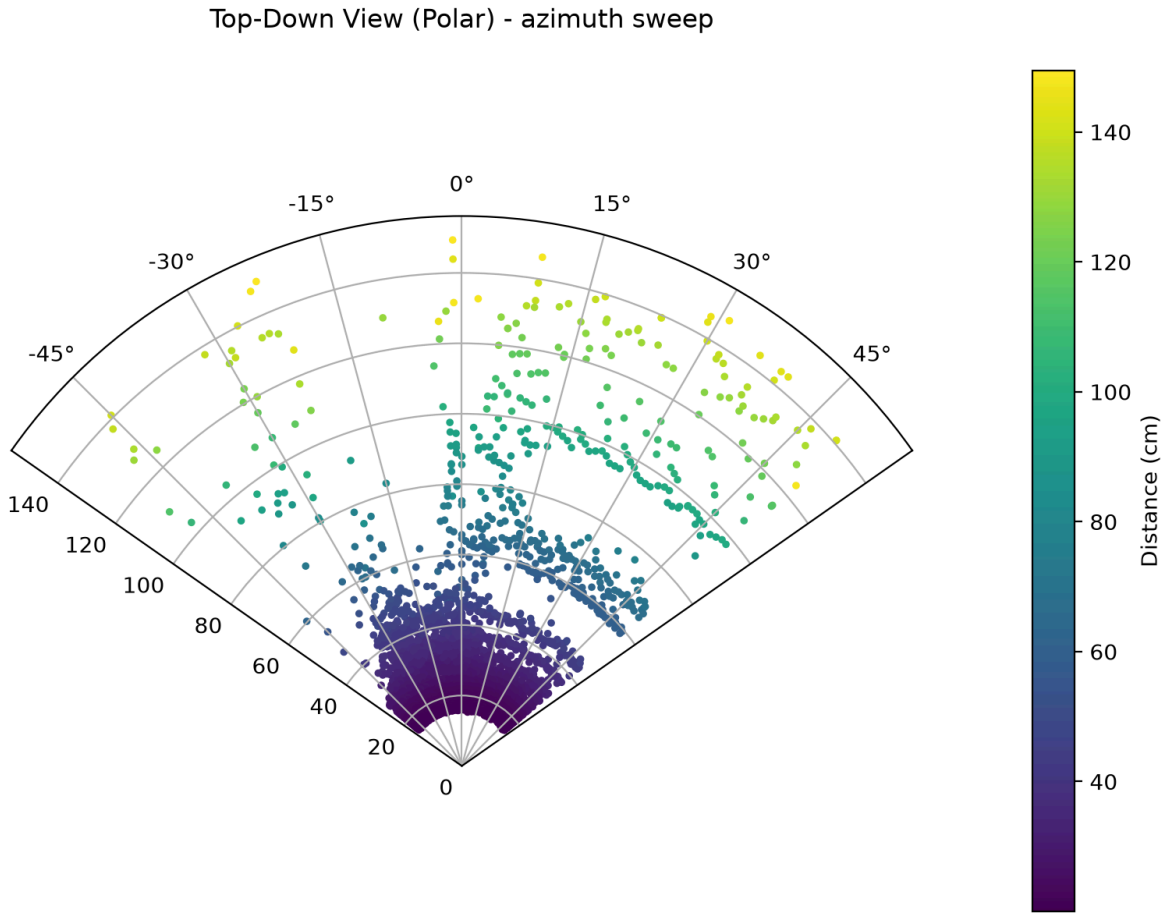
5) CA shape




(Fig. 13) Front polar view of the CA-provided crying face, filtered to under 60 cm. The face outline is visible, but most interior features blur together.




(Fig. 14) Front (X–Z) view of the CA crying face, filtered to under 60 cm. The roughly circular outline is about 25 cm across; the yellow row on the right (~58 cm) is a background surface.




(Fig. 15) Top-down polar view of the CA shape scan. The face appears at about 30–40 cm, clearly separated from the wall behind it at 70–100 cm.

To test how well our system generalized, we scanned a CA-provided crying face using the exact same sweep settings, calibration, and plotting code as our smiley face (Figs. 13–15). The overall outline came through clearly: the near-field plot shows a roughly circular face about 25 cm across, sitting about 35 cm from the scanner, and the top-down view shows it as a distinct cluster well in front of the background wall. The interior features were much harder to make out than on our smiley. One eye appears as a clear gap left of center, but the other eye, the mouth, and the tears mostly blur into the surrounding cardboard.

We think this is mainly a feature-size issue. The CA face was smaller than ours and its cutouts were narrower, so they were closer in size to the IR sensor's beam spot. When the beam straddles an edge, the sensor returns a distance somewhere between the cardboard and the wall,

which is why both scans have scattered green and yellow points along every edge instead of a clean boundary. Our smiley's eyes and mouth were large enough for the gaps to survive this blurring, while most of the CA shape's features were not.

If we continued MP2, we would filter out "mixed" edge readings that fall between the object and background distances, average more samples per point, and move the scanner closer (while staying above the 20 cm minimum) so small features take up more of the sensor's field of view.

We integrated the scan sweep by calling panServo.write() and tiltServo.write() to step through the pan and tilt ranges, taking several analogRead() samples at each point and averaging them to reduce noise before sending the result over serial. To avoid wasting time snapping the pan servo back to its starting angle before every row, we alternate the sweep direction each row (boustrophedon pattern), so the pan servo simply reverses direction at the end of each tilt row instead of resetting.

We handled distance conversion with a calibration curve: because the sensor's output voltage is approximately proportional to 1/distance, we fit distance = calibA * voltage^calibB from a set of known-distance readings using a log-linear least-squares regression, then applied that formula in the visualization script to convert every logged raw value into centimeters, discarding points outside the sensor's reliable range. Each valid point's pan/tilt angles are then treated as azimuth/elevation offsets from the scan center (pan 100°, tilt 90°) and converted to Cartesian X/Y/Z so the scan can be plotted as a point cloud.

Below are key snippets from our code that illustrate this design; the full commented listing is included in Appendix A.

**Averaged sensor read and serial data format:**

```
int readDistanceRaw() {
  long total = 0;
  for (int i = 0; i < numSamples; i++) {
    total += analogRead(sensorPin);
    delay(2);
  }
  return total / numSamples;              // average of numSamples ADC readings
}

void sendDataPoint(int panAngle, int tiltAngle, int rawValue) {
  Serial.print(panAngle);   Serial.print(",");
  Serial.print(tiltAngle);  Serial.print(",");
  Serial.println(rawValue);               // "panAngle,tiltAngle,rawValue"
}
```

This function averages several ADC readings per point to reduce sensor noise, and each result is

sent to the laptop as a comma-separated line that the Python receiver parses directly.

**Raster scan:**

```
for (int tiltAngle = tiltMin; tiltAngle <= tiltMax; tiltAngle += tiltStep) {
  tiltServo.write(tiltAngle);
  delay(settleDelay);

  bool sweepingForward = (((tiltAngle - tiltMin) / tiltStep) % 2 == 0);

  if (sweepingForward) {
    for (int panAngle = panMin; panAngle <= panMax; panAngle += panStep) {
      panServo.write(panAngle);
      delay(settleDelay);
      sendDataPoint(panAngle, tiltAngle, readDistanceRaw());
    }
  } else {
    for (int panAngle = panMax; panAngle >= panMin; panAngle -= panStep) {
      panServo.write(panAngle);
      delay(settleDelay);
      sendDataPoint(panAngle, tiltAngle, readDistanceRaw());
    }
  }
}
```

This loop steps the tilt servo down each row and alternates the pan sweep direction between rows, so the scan snakes back and forth across the grid instead of resetting to panMin before every row.

**Calibration curve and coordinate conversion:**

```
def distanceFromRaw(rawValue):
  voltage = (rawValue / adcMax) * vRef
  if voltage <= 0:
    return None
  distanceCm = calibA * math.pow(voltage, calibB)
  if distanceCm < minRange or distanceCm > maxRange:
    return None
  return distanceCm


azimuth   = math.radians(panAngle - panCenter)
elevation = math.radians(tiltAngle - tiltCenter)
x = distanceCm * math.cos(elevation) * math.sin(azimuth)
y = distanceCm * math.cos(elevation) * math.cos(azimuth)
z = distanceCm * math.sin(elevation)
```

# What worked well and what could be improved:

Mechanically, building the servo splines directly into the printed parts and aligning both rotation axes with the center of the IR sensor worked very well. It kept the part count to three prints and a base, and it meant our angle-to-coordinate math needed no offset corrections. Using one bolt size throughout also made assembly and disassembly quick. One improvement would be a higher base, since the scanner sits low and close to the table.

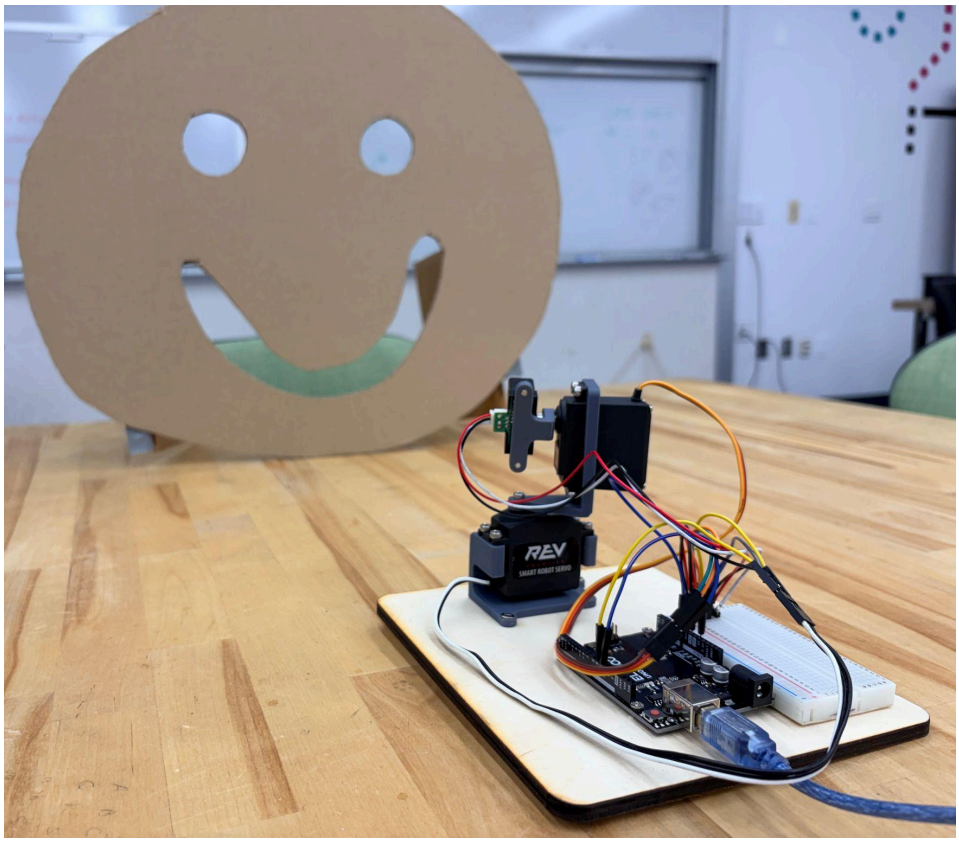
Electrically, the circuit was simple and reliable once we replaced the faulty servo. Running both servos and the sensor from the same 5V rail worked. The data was relatively noisy too and in the future we would probably opt for a capacitor for filtering.

In software, the sweep and per-point averaging gave clean, repeatable scans, and saving raw data to CSV let us re-plot and re-calibrate without rescanning. The biggest weaknesses are range and resolution, our calibration only covers distances up to 76 cm while the visualization accepts points up to 150 cm, so far-away points are extrapolated and noisy (Fig. 10), and the 1° step combined with the sensor's beam width blurs small features (Section 5). Extending the calibration range and filtering edge readings would address both.

# Integration and process:

We were able to work almost entirely in-class on this project which made any integration very easy. Since we were working together in person the whole time it was easy to stay up to date on what we were each doing. During the hardware and software integration phase we had the servo hiccup mentioned in the previous section but it was smooth sailing from there. We also found that viewing 2D scans in a polar coordinate frame had much better resolution.

Team wise we didn't really have any issues, we were both pretty unavailable outside of class but like we said above we got most everything done in class so that wasn't a problem. Communicating over Whatsapp about things like being late to class or little progress updates worked well for us.



(Fig. 16) Scanning our cardboard cutout.

# Authentic engagement discussion:

| | Aditi | Bennett |
|---|---|---|
| Mechanical Design | - Sketched initial design<br>- Printed parts<br>- Collaborated on | - CAD modeled physical parts<br>- Laser cut base |

| | | |
|---|---|---|
| | assembly | - Collaborated on assembly |
| Electronics Circuitry | - Wired servos and sensor | - Optimized the ground and power rails |
| Software Development | - Wrote majority of Arduino code<br>- Image plotting system<br>- Calibration of scan area/quality | - Initial outline/pseudo-code<br>- IR sensor calibration<br>- Servo trouble shooting |

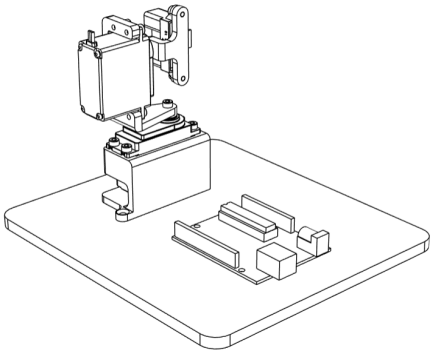
# Appendix A:

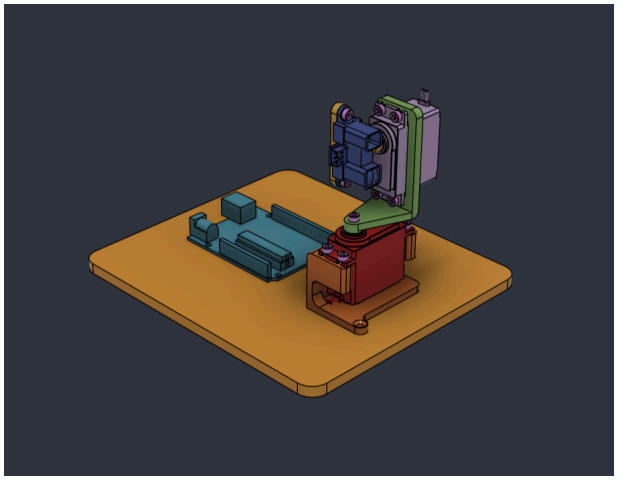
**Bill of Materials:**

| Part | Amount | Price each |
|---|---|---|
| Arduino Uno | 1 | $24.99 |
| PLA | 21g | $0.66 |
| M3 x 8mm bolt | 12 | $5.38 |
| ¼” plywood | 5in$^3$ | $1.20 |
| Servo motor | 2 | $11.99 |
| Bread board | 1 | $3.79 |
| Jumper wires | 8 | $0.05 |
| IR sensor | 1 | $5.99 |

**Items beyond the starting BOM: PLA for the printed parts, ¼" plywood base, M3 x 8mm bolts (used in place of the kit's 8-32 and 4-40 hardware), breadboard, and jumper wires. The Arduino was our own.**

**Additional CAD Images:**





(Fig. A1, left) Line render of the CAD assembly, pan servo in a printed bracket on the laser-cut plywood base, with the tilt servo and sensor holder above and the Arduino beside them. (Fig. A2, right) Color render of the same assembly, showing the three printed parts and the base.

## Source Code:

### A.1 ArduinoCalibrate.ino

```
// MP2 2.5D Scanner - calibration helper
// Holds the sensor pointed straight ahead and streams raw ADC readings
// so known-distance measurements can be taken for calibrating
// calibA/calibB in PythonVisualizeScanData.py.

#include <Servo.h>

Servo panServo;
Servo tiltServo;

const int panServoPin  = 9;
const int tiltServoPin = 10;
const int sensorPin    = A0;

const int numSamples = 10;   // more averaging than the scan sketch, since we're holding
still

int readDistanceRaw()
{
  long total = 0;
  for (int i = 0; i < numSamples; i++) {
    total += analogRead(sensorPin);
    delay(2);
  }
  return total / numSamples;
}

void setup()
{
```

```
  Serial.begin(9600);
  panServo.attach(panServoPin);
  tiltServo.attach(tiltServoPin);
  panServo.write(90);   // straight ahead
  tiltServo.write(90);  // level
  delay(500);
}

void loop()
{
  Serial.println(readDistanceRaw());
  delay(300);
}
```

## A.2 PythonCalibrationHelper.py

```
# MP2 2.5D Scanner - calibration helper
# Run this while ArduinoCalibrate.ino is uploaded to the board.
#
# Steps through a fixed list of distances (5in to 30in, step 5in).
# press Enter once the square is positioned, it records
# exactly 10 raw readings, then stops and waits to move the
# square and press Enter again for the next distance.
#
#


import csv
import serial
import sys
import time
import math

validationMode = len(sys.argv) > 1 and sys.argv[1] == "validation"

arduinoComPort = "COM3"
baudRate = 9600

serialPort = serial.Serial(arduinoComPort, baudRate, timeout=1)
time.sleep(2)

adcMax = 1023
vRef = 5.0
minValidCm = 20.0  # sensor's rated minimum reliable range

if validationMode:
  distancesIn = [12, 17, 22, 27]  # in between the fit distances
  outputFilename = "validation_data.csv"
else:
  distancesIn = [5, 10, 15, 20, 25, 30]
  outputFilename = "calibration_data.csv"
readingsPerDistance = 10

samples = []     # list of (distance_cm, avg_raw_value)
rawReadings = [] # list of (distance_in, distance_cm, raw_value), every reading

try:
  for distanceIn in distancesIn:
    input("\nPosition the square at " + str(distanceIn) + " in, then press Enter...")

    serialPort.reset_input_buffer()

    readings = []
    while len(readings) < readingsPerDistance:
      line = serialPort.readline().decode(errors="replace").strip()
      if line.isdigit():
```

```
        readings.append(int(line))

    avgRaw = sum(readings) / len(readings)
    distanceCm = distanceIn * 2.54
    print("  " + str(distanceIn) + "in (" + str(round(distanceCm, 1)) + "cm) -> avg raw = "
          + str(round(avgRaw, 1)) + " (" + str(len(readings)) + " samples)")
    samples.append((distanceCm, avgRaw))
    for r in readings:
      rawReadings.append((distanceIn, distanceCm, r))

except KeyboardInterrupt:
  print("\nStopped early by user.")

finally:
  serialPort.close()

# save every reading so the plots can be made later without re-measuring
with open(outputFilename, "w", newline="") as csvFile:
  writer = csv.writer(csvFile)
  writer.writerow(["distance_in", "distance_cm", "raw_sensor_value"])
  writer.writerows(rawReadings)
print("\nSaved " + str(len(rawReadings)) + " readings to " + outputFilename)

if validationMode:
  sys.exit(0)  # validation points must not be used in the fit

print("\nCollected points (distance_cm, raw):")
for distanceCm, avgRaw in samples:
  print("  " + str(round(distanceCm, 1)) + " cm, raw=" + str(round(avgRaw, 1)))

# fit distance = calibA * voltage^calibB  <=>  ln(distance) = ln(calibA) + calibB *
ln(voltage)
usable = [(d, r) for d, r in samples if r > 0 and d >= minValidCm]
skipped = [(d, r) for d, r in samples if not (r > 0 and d >= minValidCm)]

if skipped:
  print("\nExcluded from fit (below " + str(minValidCm) + "cm or invalid):")
  for d, r in skipped:
    print("  " + str(round(d, 1)) + " cm, raw=" + str(round(r, 1)))

if len(usable) < 2:
  print("\nNot enough valid samples to fit a curve.")
else:
  xs = [math.log((r / adcMax) * vRef) for d, r in usable]
  ys = [math.log(d) for d, r in usable]
  n = len(xs)
  meanX = sum(xs) / n
  meanY = sum(ys) / n
  b = sum((x - meanX) * (y - meanY) for x, y in zip(xs, ys)) / sum((x - meanX) ** 2 for x in
xs)
  lnA = meanY - b * meanX
  a = math.exp(lnA)
  print("\nFitted calibration (update these in PythonVisualizeScanData.py):")
  print("  calibA = " + str(round(a, 3)))
  print("  calibB = " + str(round(b, 3)))
```

### A.3 PythonPlotCalibration.py

```
# MP2 2.5D Scanner

# pip install matplotlib numpy
#
# usage:
#   python PythonPlotCalibration.py calibration_data.csv validation_data.csv
#
# both CSVs come from PythonCalibrationHelper.py
```

```
# columns: distance_in, distance_cm, raw_sensor_value
#
# Figure 1 (calibration_plot.png): sensor voltage vs. actual distance, with
#   the fitted calibration curve drawn through it.
# Figure 2 (error_plot.png): predicted distance vs. actual distance, and the
#   error, for the validation distances (NOT used in the fit).
#   If no validation file is given, it falls back to the calibration points,
#   which only shows how well the curve fits its own data.
#
# CALIBRATION FUNCTION
# The Sharp GP2Y0A02YK0F outputs a voltage that falls off roughly as
# 1/distance so we
# model it as a power law:
#
#     distance = calibA * voltage ^ calibB      (calibB should be near -1)
#
# Taking ln of both sides makes it a straight line:
#
#     ln(distance) = ln(calibA) + calibB * ln(voltage)
#
# so calibA and calibB come from an ordinary least-squares line fit in
# log-log space. Only points >= minRange are used below ~20 cm the sensor's
# output drops again, so one voltage maps to two distances and the fit breaks.

import csv
import sys

import numpy as np
import matplotlib.pyplot as plt

calibFilename = sys.argv[1] if len(sys.argv) > 1 else "calibration_data.csv"
validFilename = sys.argv[2] if len(sys.argv) > 2 else None

adcMax   = 1023
vRef     = 5.0   # ADC reference voltage
minRange = 20.0  # cm


# returns arrays of actual distance (cm) and raw reading, one per row
def loadReadings(filename):
 distances, raws = [], []
 with open(filename, newline="") as csvFile:
  reader = csv.DictReader(csvFile)
  for row in reader:
   distances.append(float(row["distance_cm"]))
   raws.append(float(row["raw_sensor_value"]))
 return np.array(distances), np.array(raws)


def voltageFromRaw(raw):
 return (raw / adcMax) * vRef


def distanceFromVoltage(voltage):
 return calibA * np.power(voltage, calibB)


# least-squares fit of ln(distance) = ln(A) + B*ln(voltage)
# returns (A, B, rSquared); rSquared is computed in log-log space
def fitPowerLaw(distances, voltages):
 lnV = np.log(voltages)
 lnD = np.log(distances)
 b, lnA = np.polyfit(lnV, lnD, 1)
 residuals = lnD - (lnA + b * lnV)
 rSquared = 1 - np.sum(residuals ** 2) / np.sum((lnD - lnD.mean()) ** 2)
```

```
 return np.exp(lnA), b, rSquared


# averages readings taken at the same distance
# returns (distances, mean, std) arrays
def groupByDistance(distances, values):
 unique = np.unique(distances)
 means = np.array([values[distances == d].mean() for d in unique])
 stds  = np.array([values[distances == d].std() for d in unique])
 return unique, means, stds



calibDist, calibRaw = loadReadings(calibFilename)
calibVolt = voltageFromRaw(calibRaw)
inFit = (calibDist >= minRange) & (calibVolt > 0)

calibA, calibB, rSquared = fitPowerLaw(calibDist[inFit], calibVolt[inFit])
print("Fitted calibration from " + calibFilename + ":")
print("  distance = " + str(round(calibA, 3)) + " * voltage^" + str(round(calibB, 3))
    + "   (R^2 = " + str(round(rSquared, 4)) + " in log-log space)")
print("  -> set calibA/calibB in PythonVisualizeScanData.py to these values")

groupDist, groupVolt, groupStd = groupByDistance(calibDist, calibVolt)
groupInFit = groupDist >= minRange

fig1, ax1 = plt.subplots(figsize=(8, 6))

# individual readings, faded, so the spread at each distance is visible
ax1.scatter(calibDist[inFit], calibVolt[inFit], s=10, alpha=0.3,
        color="tab:blue", label="Individual readings")
ax1.scatter(calibDist[~inFit], calibVolt[~inFit], s=10, alpha=0.3,
        color="tab:gray", label="Readings below " + str(int(minRange)) + " cm (not fit)")

# mean at each distance with a +/- 1 std error bar
ax1.errorbar(groupDist[groupInFit], groupVolt[groupInFit], yerr=groupStd[groupInFit],
         fmt="o", color="tab:blue", capsize=4, label="Mean ± 1 std")
ax1.errorbar(groupDist[~groupInFit], groupVolt[~groupInFit], yerr=groupStd[~groupInFit],
         fmt="o", color="tab:gray", capsize=4)

# fitted curve, drawn as voltage vs. distance
curveDist = np.linspace(minRange, max(calibDist.max(), 80), 200)
curveVolt = np.power(curveDist / calibA, 1.0 / calibB)
ax1.plot(curveDist, curveVolt, color="tab:orange", linewidth=2,
      label="Fit: d = " + str(round(calibA, 2)) + " · V^(" + str(round(calibB, 3))
          + "),  R² = " + str(round(rSquared, 3)))

ax1.set_xlabel("Actual distance (cm)")
ax1.set_ylabel("Sensor output (V)")
ax1.set_title("IR Sensor Calibration")
ax1.set_xlim(0, curveDist.max() * 1.05)
ax1.grid(alpha=0.3)
ax1.legend()
fig1.tight_layout()
fig1.savefig("calibration_plot.png", dpi=200)



if validFilename:
 errDist, errRaw = loadReadings(validFilename)
 uniqueIn = sorted(set(round(d / 2.54) for d in errDist))
```

```
 distList = ", ".join(str(d) for d in uniqueIn) + " in"
 errSource = "held-out validation set (" + distList + " - NOT used in the calibration fit)"
else:
 errDist, errRaw = calibDist[inFit], calibRaw[inFit]
 errSource = "calibration points (in-sample fit check, not true validation)"
 print("No validation file given - error plot uses the calibration points themselves.")

predicted = distanceFromVoltage(voltageFromRaw(errRaw))
error = predicted - errDist

groupErrDist, groupPred, groupPredStd = groupByDistance(errDist, predicted)
groupErr = groupPred - groupErrDist

fig2, (axPred, axErr) = plt.subplots(1, 2, figsize=(12, 5))

# predicted vs actual
lineMax = max(errDist.max(), predicted.max()) * 1.1
axPred.plot([0, lineMax], [0, lineMax], "--", color="gray", label="Ideal (predicted = actual)")
axPred.scatter(errDist, predicted, s=10, alpha=0.3, color="tab:blue", label="Individual readings")
axPred.errorbar(groupErrDist, groupPred, yerr=groupPredStd, fmt="o",
          color="tab:blue", capsize=4, label="Mean ± 1 std")
axPred.set_xlim(0, lineMax)
axPred.set_ylim(0, lineMax)
axPred.set_aspect("equal", adjustable="box")
axPred.set_xlabel("Actual distance (cm)")
axPred.set_ylabel("Predicted distance (cm)")
axPred.set_title("Predicted vs. Actual")
axPred.grid(alpha=0.3)
axPred.legend()

# error at each distance
axErr.axhline(0, color="gray", linestyle="--")
axErr.scatter(errDist, error, s=10, alpha=0.3, color="tab:red", label="Individual readings")
axErr.errorbar(groupErrDist, groupErr, yerr=groupPredStd, fmt="o",
          color="tab:red", capsize=4, label="Mean ± 1 std")
axErr.set_xlabel("Actual distance (cm)")
axErr.set_ylabel("Error: predicted - actual (cm)")
axErr.set_title("Measurement Error")
axErr.grid(alpha=0.3)
axErr.legend()

fig2.suptitle("Calibration Error - " + errSource)
fig2.tight_layout()
fig2.savefig("error_plot.png", dpi=200)

# print a summary for write-up
print("\nError summary (" + errSource + "):")
for d, e, s in zip(groupErrDist, groupErr, groupPredStd):
 print("  " + str(round(d, 1)) + " cm: mean error = " + str(round(e, 2))
     + " cm (" + str(round(100 * e / d, 1)) + "%), std = " + str(round(s, 2)) + " cm")
print("  overall mean absolute error = " + str(round(np.abs(error).mean(), 2)) + " cm")

print("\nSaved calibration_plot.png and error_plot.png")

plt.show()
```

### A.4 ArduinoScanSweep.ino

```
// MP2 2.5D Scanner

#include <Servo.h>

Servo panServo;               // sweeps left/right
Servo tiltServo;              // sweeps up/down

const int panServoPin  = 9;
const int tiltServoPin = 10;
const int sensorPin    = A0;  // analog output

// sweep range/step settings TUNE LATER
// units are degrees, matching Servo.write()
const int panMin   = 50;
const int panMax   = 150;
const int panStep  = 1;    // Servo.write() only has 1-degree resolutio
const int tiltMin  = 50;
const int tiltMax  = 130;
const int tiltStep = 1;    // same as panStep

const int settleDelay = 150;   // ms
const int numSamples   = 5;    // number of ADC readings averaged per point


void setup()
{
  long baudRate = 9600;        // must match the baud rate in the Python script
  Serial.begin(baudRate);

  panServo.attach(panServoPin);
  tiltServo.attach(tiltServoPin);

  panServo.write(panMin);
  tiltServo.write(tiltMin);
  delay(500);                  // give servos time to reach the starting position
}


// take several ADC readings and average them to reduce noise
int readDistanceRaw()
{
  long total = 0;
  for (int i = 0; i < numSamples; i++) {
    total += analogRead(sensorPin);
    delay(2);
  }
  return total / numSamples;
}



// send one data point back to the laptop as "panAngle,tiltAngle,rawValue"
void sendDataPoint(int panAngle, int tiltAngle, int rawValue)
{
  Serial.print(panAngle);   Serial.print(",");
  Serial.print(tiltAngle);  Serial.print(",");
  Serial.println(rawValue);
}


// raster-scan the sensor across the pan/tilt range one time
// then stop (so the scan doesn't repeat and confuse the data being collected)
void loop()
{
  for (int tiltAngle = tiltMin; tiltAngle <= tiltMax; tiltAngle += tiltStep) {
    tiltServo.write(tiltAngle);
    delay(settleDelay);

    // alternate sweep direction each row so we don't waste
    // time snapping the pan servo back to panMin before every row
```

```
    bool sweepingForward = (((tiltAngle - tiltMin) / tiltStep) % 2 == 0);

    if (sweepingForward) {
      for (int panAngle = panMin; panAngle <= panMax; panAngle += panStep) {
        panServo.write(panAngle);
        delay(settleDelay);
        int rawValue = readDistanceRaw();
        sendDataPoint(panAngle, tiltAngle, rawValue);
      }
    } else {
      for (int panAngle = panMax; panAngle >= panMin; panAngle -= panStep) {
        panServo.write(panAngle);
        delay(settleDelay);
        int rawValue = readDistanceRaw();
        sendDataPoint(panAngle, tiltAngle, rawValue);
      }
    }
  }

  while (true) {
    delay(1000);
  }
}
```

### A.5 PythonReceiveScanData.py

```
# MP2 2.5D Scanner

import serial  # pyright: ignore[reportMissingModuleSource]
import csv
import time

# sends one line per data point formatted as "panAngle,tiltAngle,rawValue"

arduinoComPort = "COM3"
baudRate = 9600

serialPort = serial.Serial(arduinoComPort, baudRate, timeout=1)
time.sleep(2)  # give the Arduino time to reset after the serial port opens

outputFilename = "scan_data.csv"

scanData = []  # list of (panAngle, tiltAngle, rawSensorValue) tuples

print("receiving scan data. press Ctrl+C to quit.")

try:
  while True:
    lineOfData = serialPort.readline().decode().strip()

    if len(lineOfData) == 0:
      continue

    # convert the line into 3 integers; skip any partial line
    try:
      panAngle, tiltAngle, rawValue = (int(x) for x in lineOfData.split(','))
    except ValueError:
      continue

    scanData.append((panAngle, tiltAngle, rawValue))
    print("pan = " + str(panAngle), end="")
    print(", tilt = " + str(tiltAngle), end="")
    print(", raw = " + str(rawValue))

except KeyboardInterrupt:
```

```
  print("Stopped by user.")

finally:
  serialPort.close()

  # save everything collected so far to a CSV for calibration/visualization later
  with open(outputFilename, "w", newline="") as csvFile:
    writer = csv.writer(csvFile)
    writer.writerow(["pan_angle_deg", "tilt_angle_deg", "raw_sensor_value"])
    writer.writerows(scanData)

  print("Saved " + str(len(scanData)) + " points to " + outputFilename)
```

### A.6 PythonVisualizeScanData.py

```
# pip install matplotlib numpy

import csv
import math
import sys

import numpy as np
import matplotlib.pyplot as plt

# reads the CSV written by PythonReceiveScanData.py
# columns: pan_angle_deg, tilt_angle_deg, raw_sensor_value
#
# usage: python PythonVisualizeScanData.py scan_data.csv
# saves <csv name>_front.png and <csv name>_topdown.png next to the CSV,
# so scanning the CA shape to a different CSV gives separate images

inputFilename = sys.argv[1] if len(sys.argv) > 1 else "scan_data.csv"

# calibration: raw ADC reading -> distance in cm
# The sensor outputs a voltage that is bascially proportional to 1/distance.


# TUNE THIS

adcMax    = 1023
vRef      = 5.0     # runs adc off of 5V
calibA    = 61.211  # fitted 2026-09-28 from calibration_data.csv (R^2 = 0.9913)
calibB    = -1.03
minRange  = 20.0    # cm, below this the sensor's output is ambiguous
maxRange  = 150.0   # cm, above this the sensor's output is unreliable

# straiht ahead servo angles  must match the midpoint of panMin/panMax and
# tiltMin/tiltMax in ArduinoScanSweep.ino
panCenter  = 100.0  # (panMin + panMax) / 2 = (50 + 150) / 2
tiltCenter = 90.0   # (tiltMin + tiltMax) / 2 = (50 + 130) / 2

# takes a raw adc value and turns it into a distance based
# on the formula and returns it if it is valid
def distanceFromRaw(rawValue):
  voltage = (rawValue / adcMax) * vRef
  if voltage <= 0:
    return None
  distanceCm = calibA * math.pow(voltage, calibB)
  if distanceCm < minRange or distanceCm > maxRange:
    return None
  return distanceCm
```

```
# 4 empty lists to collect x y and z coords of every valid point
xs, ys, zs, dists = [], [], [], []
azimuths, elevations = [], []  # radians, kept for the polar plots
skippedOutOfRange = 0

# open csv and read every row as a dictionary so we can get
# columns by name instead of position
with open(inputFilename, newline="") as csvFile:
  reader = csv.DictReader(csvFile)
  for row in reader:
    # convert text to numbers
    panAngle  = float(row["pan_angle_deg"])
    tiltAngle = float(row["tilt_angle_deg"])
    rawValue  = float(row["raw_sensor_value"])

    # dont plot if out of range
    distanceCm = distanceFromRaw(rawValue)
    if distanceCm is None:
      skippedOutOfRange += 1
      continue

    # angles of how off center sensor is
    azimuth   = math.radians(panAngle - panCenter)
    elevation = math.radians(tiltAngle - tiltCenter)

    # convert distance and angles to catesian coords
    xs.append(distanceCm * math.cos(elevation) * math.sin(azimuth))
    ys.append(distanceCm * math.cos(elevation) * math.cos(azimuth))
    zs.append(distanceCm * math.sin(elevation))
    # save distance for plot
    dists.append(distanceCm)
    azimuths.append(azimuth)
    elevations.append(elevation)

# sanity check to see how many points were saved and skipped
# help us see if our calibration range is reasonable
print("Loaded " + str(len(dists)) + " points ("
      + str(skippedOutOfRange) + " skipped as out of sensor range).")

if len(dists) == 0:
  print("Nothing to plot.")
  sys.exit(1)

# convert to numpy arrays for easier math
xs, ys, zs, dists = np.array(xs), np.array(ys), np.array(zs), np.array(dists)
azimuths, elevations = np.array(azimuths), np.array(elevations)

# plot
# depth (Y - forward) is dropped as a spatial axis and shown as color instead,
# so this is a 2D scatter over X (left/right) and Z (up/down) - like a range image
fig, ax = plt.subplots(figsize=(9, 7))

# sequential colormap
# plot every point in 2d space (left/right, up/down), colored by distance
scatter = ax.scatter(xs, zs, c=dists, cmap="viridis", s=12)

# label plot
colorBar = fig.colorbar(scatter, ax=ax, shrink=0.8, pad=0.03)
colorBar.set_label("Distance (cm)")

ax.set_xlabel("X - left/right (cm)")
ax.set_ylabel("Z - up/down (cm)")
ax.set_title("3D Scan Point Cloud (depth shown as color)")

# equal aspect ratio so the shape isn't visually distorted
ax.set_aspect("equal", adjustable="box")

# clean layout
plt.tight_layout()
```

```
# top-down view: X (left/right) vs Y (forward depth)
# the shape shows up as a band of points closer than the wall behind it
figTop, axTop = plt.subplots(figsize=(9, 6))
axTop.scatter(xs, ys, c=dists, cmap="viridis", s=6)
axTop.scatter([0], [0], marker="^", color="black", s=80, label="Scanner")
axTop.set_xlabel("X - left/right (cm)")
axTop.set_ylabel("Y - forward depth (cm)")
axTop.set_title("Top-Down View (shape vs. background)")
axTop.set_aspect("equal", adjustable="box")
axTop.legend()
figTop.tight_layout()

# polar versions of the same two views, for comparison against the
# rectangular ones above
#
# top-down polar: theta = azimuth (pan offset from center), r = horizontal
# distance (= dist * cos(elevation), i.e. distance projected flat, ignoring
# height) - same data as the rectangular top-down view, just in polar form
horizDist = np.sqrt(xs ** 2 + ys ** 2)

# side polar: theta = elevation (tilt offset from center), r = vertical-plane
# distance (= dist * cos(azimuth)) - the elevation-sweep analog of the
# top-down polar plot above
vertDist = np.sqrt(ys ** 2 + zs ** 2)


def polarWedgePlot(theta, r, title, rMax=None):
  fig, ax = plt.subplots(figsize=(8, 8), subplot_kw={"projection": "polar"})
  sc = ax.scatter(theta, r, c=dists, cmap="viridis", s=6)
  ax.set_theta_zero_location("N")   # straight ahead points "up"
  ax.set_theta_direction(-1)        # positive angle sweeps to the right
  # zoom the wedge to just the angular range actually scanned, with a
  # little padding, instead of showing a mostly-empty full circle
  thetaMinDeg = math.degrees(theta.min()) - 5
  thetaMaxDeg = math.degrees(theta.max()) + 5
  ax.set_thetamin(thetaMinDeg)
  ax.set_thetamax(thetaMaxDeg)
  if rMax is not None:
    ax.set_rmax(rMax)              # crop out the far wall so the near shape fills the plot
  ax.set_title(title)
  cb = fig.colorbar(sc, ax=ax, shrink=0.7, pad=0.1)
  cb.set_label("Distance (cm)")
  fig.tight_layout()
  return fig

zoomRadiusCm = 70.0

figTopPolar = polarWedgePlot(azimuths, horizDist, "Top-Down View (Polar) - azimuth sweep",
rMax=zoomRadiusCm)
figSidePolar = polarWedgePlot(elevations, vertDist, "Side View (Polar) - elevation sweep",
rMax=zoomRadiusCm)

# theta = compass angle around the boresight (0 deg = pan right & level,
# 90 deg = straight up) r = angular distance from dead cente

compassTheta = np.arctan2(elevations, azimuths)
angularRadius = np.sqrt(azimuths ** 2 + elevations ** 2)

figFrontPolar, axFrontPolar = plt.subplots(figsize=(8, 8), subplot_kw={"projection":
"polar"})
scFront = axFrontPolar.scatter(compassTheta, angularRadius, c=dists, cmap="viridis", s=6)
axFrontPolar.set_rmax(angularRadius.max() * 1.05)
axFrontPolar.set_title("Front View (Polar) - angle from center vs. compass direction")
cbFront = figFrontPolar.colorbar(scFront, ax=axFrontPolar, shrink=0.7, pad=0.1)
cbFront.set_label("Distance (cm)")
figFrontPolar.tight_layout()

# save images for the write-up
baseName = inputFilename.rsplit(".", 1)[0]
```

```
fig.savefig(baseName + "_front.png", dpi=200)
figTop.savefig(baseName + "_topdown.png", dpi=200)
figTopPolar.savefig(baseName + "_topdown_polar.png", dpi=200)
figSidePolar.savefig(baseName + "_side_polar.png", dpi=200)
figFrontPolar.savefig(baseName + "_front_polar.png", dpi=200)
print("Saved " + baseName + "_front.png, " + baseName + "_topdown.png, "
      + baseName + "_topdown_polar.png, " + baseName + "_side_polar.png and "
      + baseName + "_front_polar.png")

plt.show()
```